

Программирование как раздел информатики

- **Программирование** — раздел информатики, изучающий теорию, методы и технологию разработки программного обеспечения для электронных вычислительных машин (ЭВМ).
- В узком смысле **Программирование** — этап решения какой-либо задачи с помощью ЭВМ, ограниченный разработкой и отладкой программ, или же собственно процесс создания программ.

Алгоритм

- **Алгоритм** – это точное предписание, которое определяет процесс, ведущий от исходных данных к требуемому конечному результату.
- Применительно к ЭВМ **алгоритм** определяет вычислительный процесс, начинающийся с обработки

Основные свойства алгоритмов

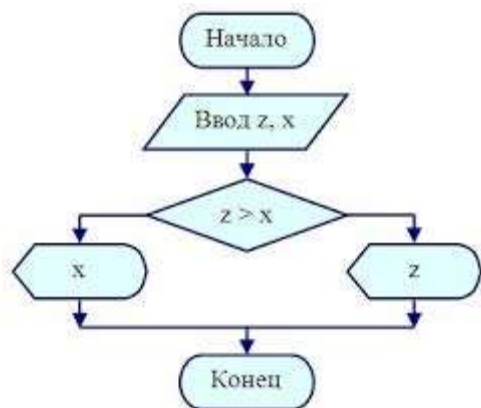
- Результативность
- Определенность
- Массовость
- Дискретность

Задание алгоритма

- совокупность возможных исходных данных, промежуточных и конечных результатов;
- правило начала;
- правило непосредственной переработки информации;
- правило окончания;
- правило извлечения результатов.

Способы описания алгоритма

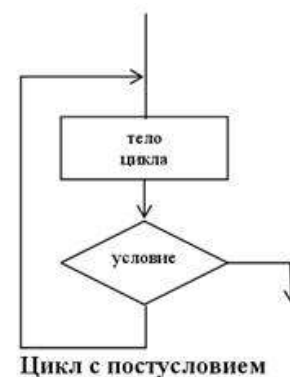
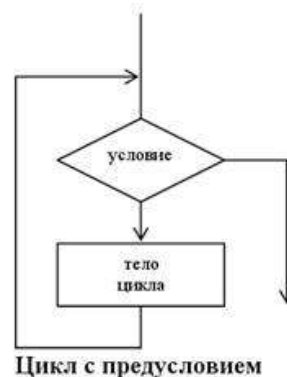
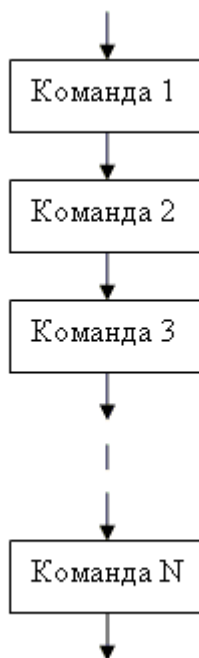
- Словесный
- Формульно-словесный
- Графический
- Программный



```
procedure InsertSymbol(H: HWND; Code: WPARAM);  
var  
    Symbol: WideChar;  
    DC: HDC;  
begin  
    with EditInfoPtr(S) do begin  
        DC := GetDC(H);  
        if Length(S) = 0 then begin  
            SetLength(S, 1);  
            Caret.Line := 0;  
        end;  
        if (Caret.Line = Low(S)) and (Caret.Line < High(S)) then begin  
            MultiByteToWideChar(CP_THREAD_ACP, 0, @Code, 1, @Symbol, 1);  
            InsertToStream(S[Caret.Line], Caret.Symbol, Symbol);  
            Larged(H, DC, 3[Caret.Line]);  
            Inc(Caret.Symbol);  
        end;  
        ReleaseDC(H, DC);  
    end;  
end;
```

Основные алгоритмические конструкции

- Следование
- Ветвление
- Цикл



Компиляция и интерпретация

- **Интерпретатор** – это программа, которая получает исходную программу и по мере распознавания конструкций входного языка реализует действия, описываемые этими конструкциями.
- **Транслятор** – принимает на вход исходную программу и порождает программу, записанную на объектном языке программирования (объектный код).
- Транслятор с языка высокого уровня называют **компилятором**.

Методологии программирования

- **Методология программирования** — совокупность методов, применяемых на различных стадиях жизненного цикла программного обеспечения и имеющих общий философский подход.

Каждая методология характеризуется своим:

- философским подходом или основными принципами. Эти принципы, от которых зависит эффективность всей методологии, обычно можно кратко сформулировать и легко объяснить;
- согласованным множеством моделей методов, которые реализуют данную методологию;
- концепциями (понятиями), позволяющими более точно определить методы.

В частном случае, когда методология применяется на стадии программирования (конструирования), её обычно называют **парадигмой программирования**.

Классификация программирования по ядрам методологии:

- Императивное программирование
- Функциональное программирование
- Логическое программирование
- Объектно-ориентированное
программирование

Императивное программирование

- программа = последовательность действий, связанных условными и безусловными переходами

```
Procedure Вскипятить_чайник
begin
    Зажечь плиту;
    Взять чайник;
    Налить в чайник воды;
    Поставить на плиту;
    Подождать 5 минут;
end
begin
    if Чайник не пуст then
        Вылить из чайника воду;
    Вскипятить_чайник;
end.
```

Функциональное программирование

- программа = система определений функций, описание того, что нужно вычислить, а как это сделать — решает транслятор; последовательность действий не прослеживается
- Примеры языков: Haskell, LISP



Логическое программирование

- программа = система определений и правил вида "условие => новый факт"
- Примеры языков: Visual Prolog, Oz и Fril

Visual Prolog



Объектно-ориентированное программирование

- программа = несколько взаимодействующих объектов, функциональность (действия) и данные распределяются между этими объектами

Объектно-ориентированное программирование

```
class Плита {  
    Горит Ли Конфорка? (конфорка)  
  
    Зажечь Конфорку (конфорка);  
  
    Потушить Конфорку (конфорка);  
  
    Установить Уровень Нагрева (конфорка, уровень);  
}
```

Интерфейс чайника может быть, например, таким:

```
class Чайник {  
    // boolean  
    Пустой ли Чайник();  
  
    // boolean  
    В Процессе Нагрева();  
  
    // Возвращает boolean (удалось или нет)  
    Поставить На Плиту(плита, конфорка);  
}
```

Свойства парадигм программирования

Парадигма	Ключевой концепт	Программа	Выполнение программы	Результат
Императивная	Команда	Последовательность команд	Исполнение команд	Итоговое состояние памяти
Функциональная	Функция	Набор функций	Вычисление функций	Значение главной функции
Логическая	Предикат	Логические формулы	Логическое доказательство	Результат доказательства
Объектно-ориентированная	Объект	Набор классов объектов	Обмен сообщениями между объектами	Результирующее состояние объектов

Этапы решения задачи на компьютере

1. Постановка задачи
2. Анализ и исследование задачи, модели
3. Разработка алгоритма
4. Программирование
5. Тестирование и отладка
6. Анализ результатов решения задачи и уточнение в случае необходимости математической модели с повторным выполнением этапов 2-5.
7. Сопровождение программы